数字逻辑基础与Verilog设计
---组合篇

第4章 组合电路模块

本章内容

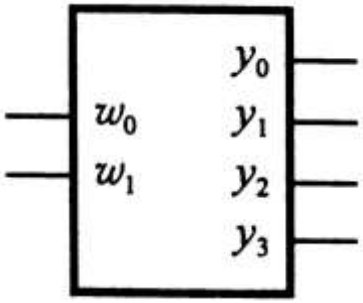
- 多路选择器（4.1）
- 译码器、多路分配器、奇偶校验电路（4.2）
  - 用Verilog设计二进制译码器（4.6）
- 编码器、代码转换器与算术比较电路（4.3-4.5）
- 算术运算电路（3.4，…）

3.译码器

工作原理：

将输入的二进制代码转换成特定的输出信号的电路。

- 输入：二进制代码（N位）
- 输出：2^N个，每个输出对应一个最小项。
- 最简单是译码器是2线-4线译码器。4个输出端分别对应其中一种输入状态。因此，又把2位二进制输入的译码器称为2线—4线译码器（ 2-to-4 decoder）。



b）图形符号

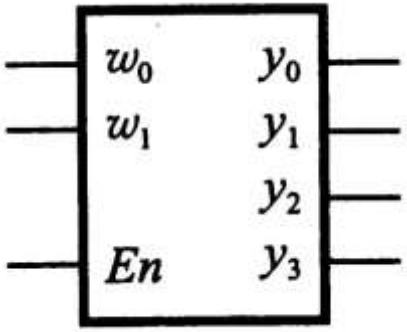
w_1	w_0	y_0	y_1	y_2	y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

a）真值表

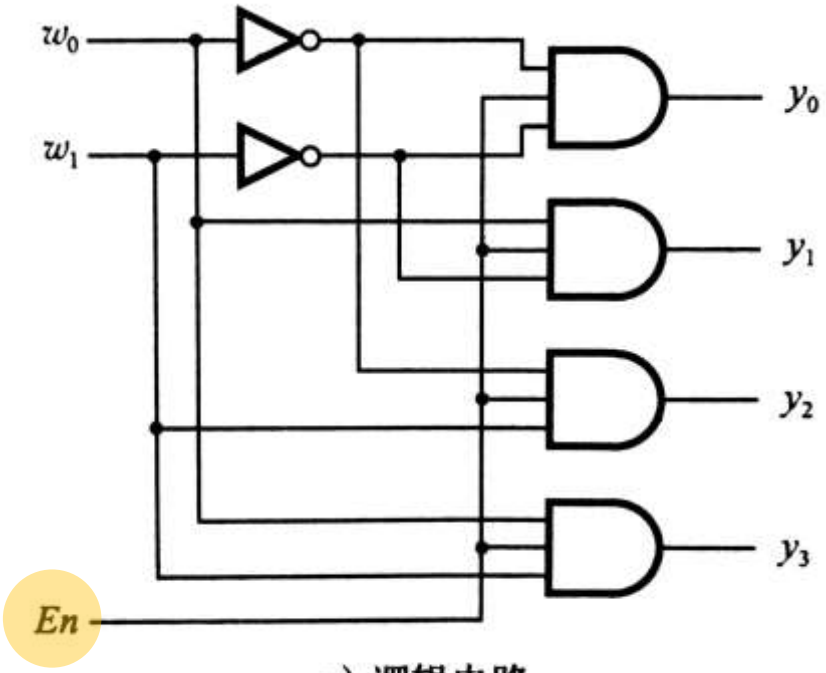
3.译码器

En	w_1	w_0	y_0	y_1	y_2	y_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) 真值表



(b) 2-to4译码器图形符号



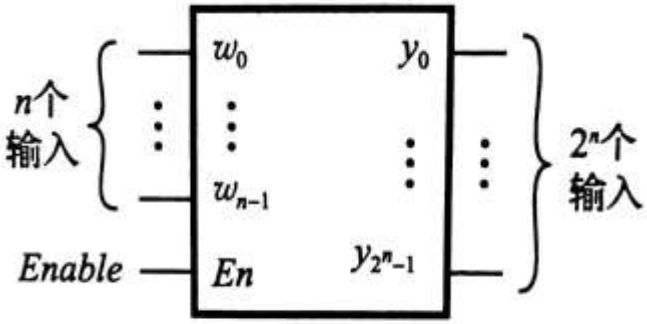
c）逻辑电路

(c) 电路符号图

3.译码器

- 带有使能端 *En* 的二进制译码器：
  - *En*=0：译码器不工作，所有输出均为0.
  - *En*=1：若以高电平作为译码器输出的有效电平，则 $y_i = m_i$（i =0, ···, n-1），称为最小项译码器。

- 若二进制码中有且只有1位取值为1，称为独热码；二进制译码器的输出属于独热码.



(d) An n-to-2^n 译码器

3.译码器

二进制译码器的应用：

- 利用小型译码器构建大型的译码器；
- 利用译码器为元件进行逻辑综合；
- 数据分配器；
- •••

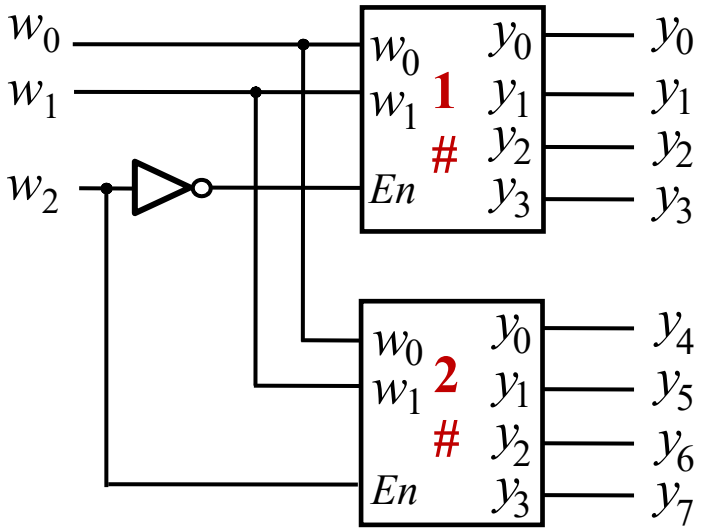
3.译码器

利用小型译码器构建大型的译码器：

例9：用2-4译码器构建3-8译码器

分析：3-8译码器有3个地址端，而2-4译码器只有2个，需利用使能端*En*进行扩展。

- 1#译码器$En = w'_2$
- 2#译码器$En = w_2$：
  - 1#译码器对二进制地址000-011解码。
  - 2#译码器对二进制地址100-111解码。



3.译码器

利用小型译码器构建大型的译码器：

例9：用2-4译码器构建3-8译码器（接）

•增加使能端S：1#译码器$En=S\cdot w'_2$，2#译码器$En=S'\cdot w_2$

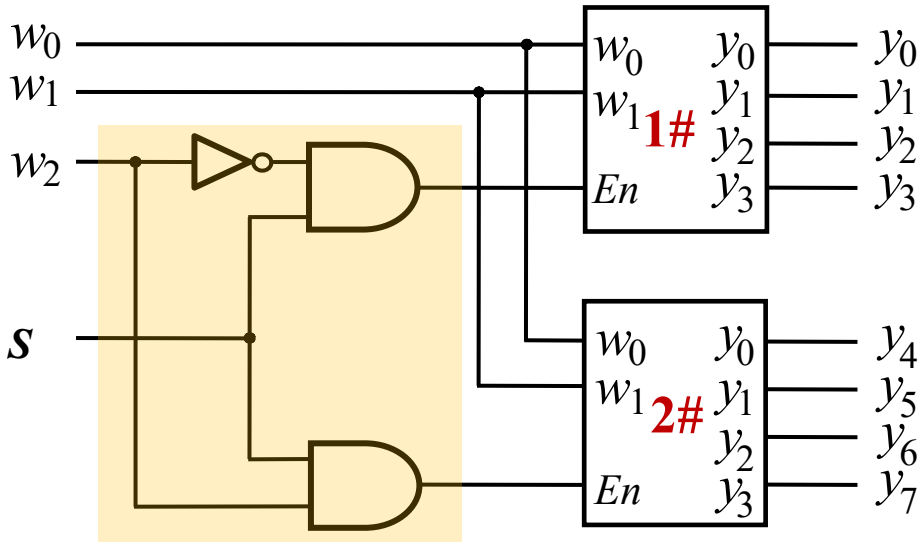


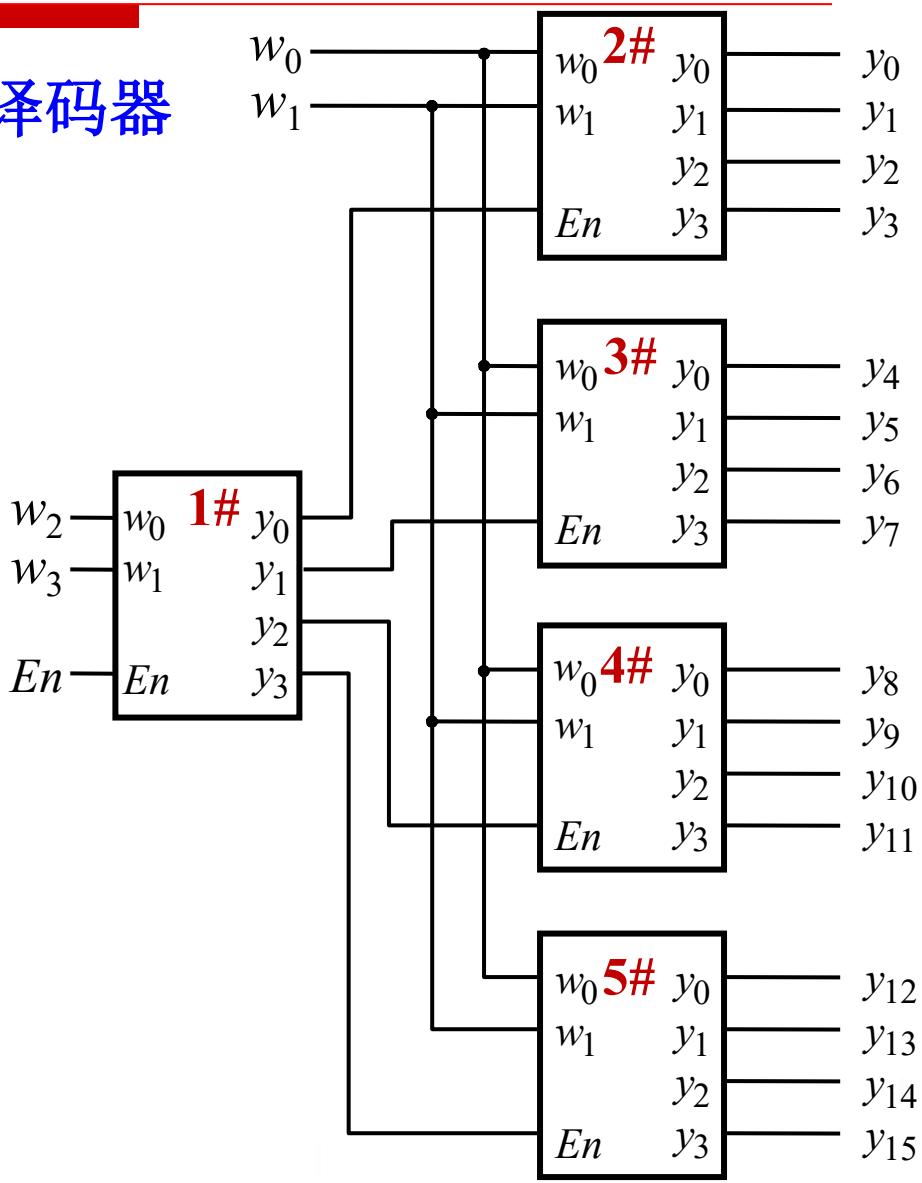
图 4.15 2-4 译码器实现3-8译码器

3.译码器

例10：采用译码器树构建4-16译码器

分析：选用2-4译码器构建

- ***En*=0:1#译码器输出全为0，所有译码器不正常工作.**
- ***En*=1:1#译码器工作，2~5#译码器按地址进行解码，如**
  - **当w_3w_2=01时, 3#译码器工作，对二进制地址0100-0111解码。**



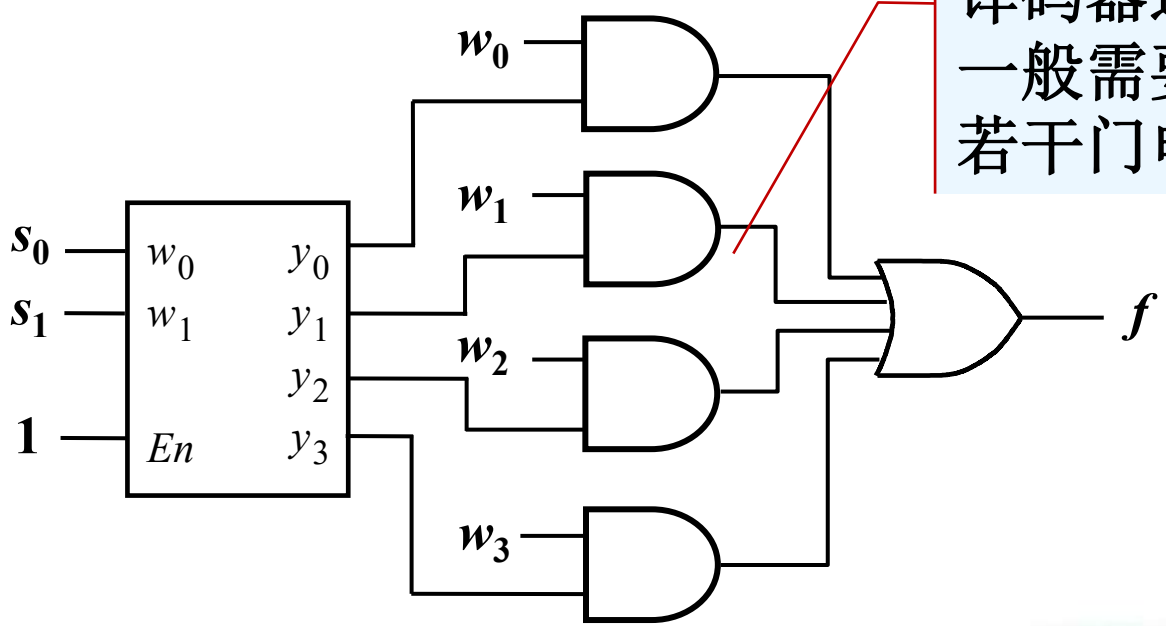
3.译码器

以译码器为元件的逻辑综合：

例11：采用2-4译码器构建4-1MUX

$$f = s'_1 s'_0 w_0 + s'_1 s_0 w_1 + s_1 s'_0 w_2 + s_1 s_0 w_3$$

$$= y_0 w_0 + y_1 w_1 + y_2 w_2 + y_3 w_3$$



译码器逻辑综合一般需要借助于若干门电路。

3.译码器

以译码器为元件的逻辑综合：

例12：采用3-8译码器实现三输入“择多”函数

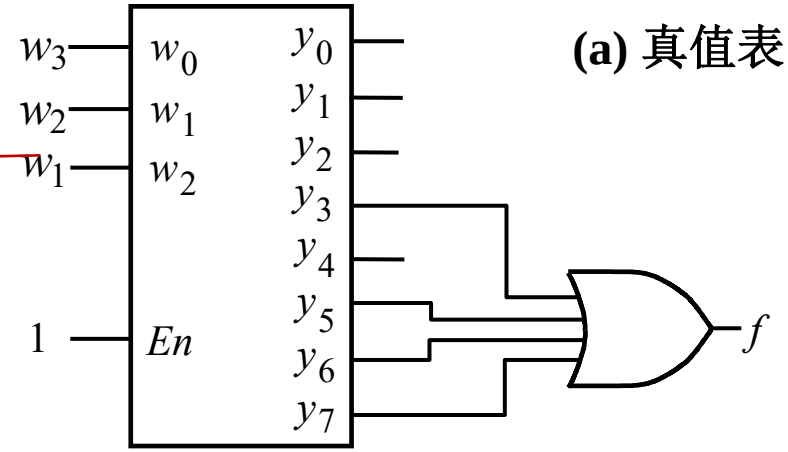
分析：译码器输出为最小项，可以实现任意函数

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(a) 真值表

$$f = w'_1w_2w_3+w_1w'_2w_3+w_1w_2w'_3+w_1w_2w_3$$
$$= m_3+m_5+m_6+m_7$$
$$= y_3 + y_5 + y_6 + y_7$$

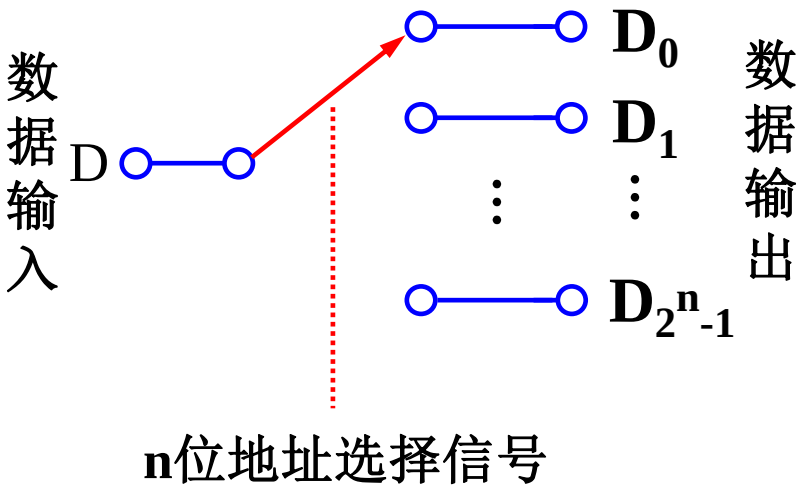
3-8译码器+或门可以综合任意3变量的逻辑函数。



3.译码器

以译码器构建数据分配器：

数据分配器：将一路输入数据根据地址选择码分配给多路数据输出中的某一路输出。



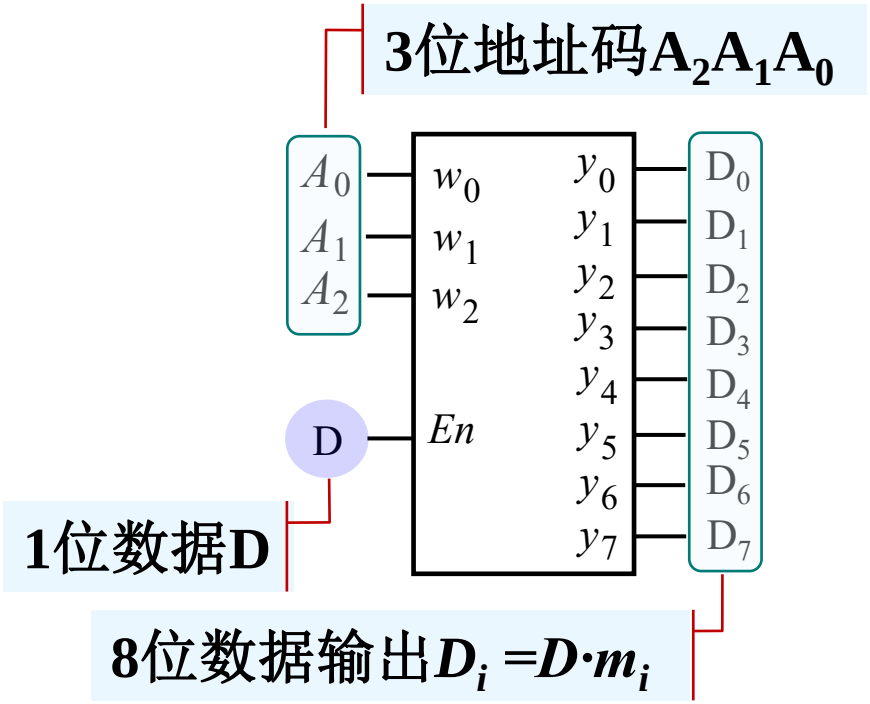


Figure. A 1-to-8 demultiplexer built using a 3-to-8 decoder.

***n*-2^{n}译码器可以用作1-*n*数据分配器.**

4.用Verilog设计译码器

主要从行为级描述组合逻辑电路：

- 主要知识要点：
  - 运算符
  - 用case语句对译码器建模；
  - 循环语句：for语句；

4.用Verilog设计译码器

（1）运算符

向量：A[2:0], B[2:0], C[2:0]
标量：f；

按位运算，eg:

C=A&B;//C[i]=A[i]·B[i]

f = A|B; // f=A[0]+B[0]

逻辑运算：对矢量运算时，先按位“或”,再对应运算，结果为标量. eg:f =A&&B;

缩减运算:单个向量操作数运算，缩为1位二进制数.

eg: f =&B;//f=B[0]·B[1]·B[2]

Table 4.2. Verilog operators.

Operator type	Operator symbols	Operation performed	Number of operands
Bitwise	~	1's complement	1
	&	Bitwise AND	2
	|	Bitwise OR	2
	^	Bitwise XOR	2
	~^ or ^~	Bitwise XNOR	2
Logical	!	NOT	1
	&&	AND	2
	||	OR	2
Reduction	&	Reduction AND	1
	~&	Reduction NAND	1
	|	Reduction OR	1
	~|	Reduction NOR	1
	^	Reduction XOR	1
	~^ or ^~	Reduction XNOR	1

4.用Verilog设计译码器

算术运算 eg:
C=A+B;
//A和B三位和赋给C

关系运算: 输出逻辑值1或0

等式运算：任何一个操作数为x或z，结果位x.

移位运算：空位用0填补

位连接运算：形成更大的矢量.

复制运算，对同一个矢量进行重复连接.

Arithmetic	+ − − * /	Addition Subtraction 2's complement Multiplication Division	2 2 1 2 2
Relational	>> < >= <=	Greater than Less than Greater than or equal to Less than or equal to	2 2 2 2
Equality	== !=	Logical equality Logical inequality	2 2
Shift	>> <<	Right shift Left shift	2 2
Concatenation	{,}	Concatenation	Any number
Replication	{{}}	Replication	Any number
Conditional	?:	Conditional	3

4.用Verilog设计译码器

（1）运算符

- 由上到下，优先级降低；同行优先级相同；
- 用括号消除可能的误解，代码变得更清晰易读。

Operator type	Operator symbols	Precedence
Complement	! ~ –	Highest precedence
Arithmetic	* / + –	
Shift	<< >>	
Relational	< <= > >=	
Equality	== !=	
Reduction	& ~& ^ ~^ | ~|	
Logical	&& ||	
Conditional	?:	Lowest precedence

4.用Verilog设计译码器

（2）采用case语句对 2-to-4 decoder建模

```
module dec2to4 (
    input   [1:0] w,
    input    en,
    output  reg [0:3]  y
);
    always @(w,en)
      case ({en,w})
        3'b100: y = 4'b1000;
        3'b101: y = 4'b0100;
        3'b110: y = 4'b0010;
        3'b111: y = 4'b0001;
        default: y = 4'b0000;
      endcase
endmodule
```

```
module dec2to4 (
    input   [1:0] w,
    input    en,
    output  reg  [0:3] y
);
    always @(w,en)
    if (en==0)    y=4'b0000;
    else
        case (w)
            2'b00: y = 4'b1000;
            2'b01: y = 4'b1000;
            2'b10: y = 4'b1000;
            2'b11: y = 4'b1000;
        endcase
endmodule
```

4.用Verilog设计译码器

- 采用层次化设计，实现 4-to-16 decoder

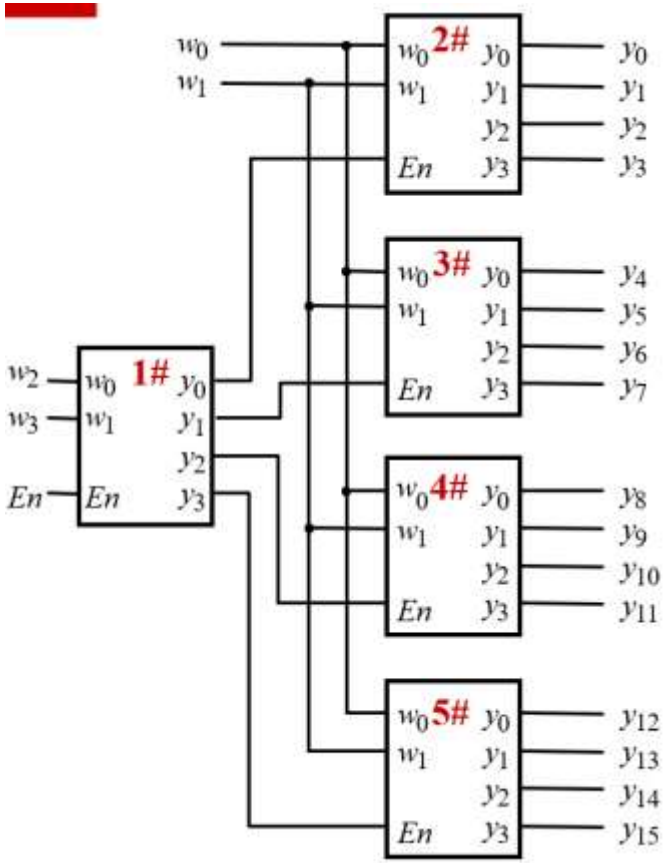


Figure 4.16 A 4-to-16 decoder

```
module dec4to16 (
        input   [3:0] w,
        input    en,
        output  [0:15] y
);
        wire [0:3] m;

        dec2to4  dec1 (w[3:2], en, m[0:3]);
        dec2to4  dec2 (w[1:0], m[0] , y[0:3] );
        dec2to4  dec3 (w[1:0], m[1] , y[4:7] );
        dec2to4  dec4 (w[1:0], m[2] , y[8:11]);
        dec2to4  dec5 (w[1:0], m[3], y[12:15]);

endmodule
```

4.用Verilog设计译码器

- 采用层次化设计，实现 4-to-1 MUX

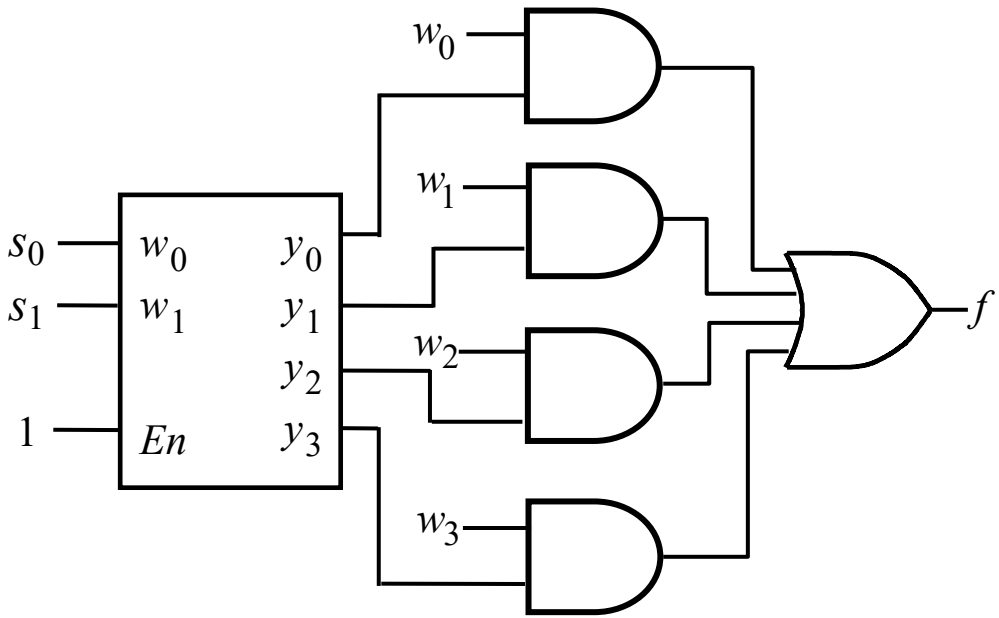


图4.17 译码器实现 4-1 mux

```
module mux4to1 (
        input   [0:3] w,
        input   [1:0] s,
        output    f
);
        wire [0:3] y;
        dec2to4 decoder (s,1,y);
        assign f =|(w&y);
       //&按位“与”,|缩减“或”
endmodule
```

图 4.52 层次化设计4-1 mux

4.用Verilog设计译码器

(3) for循环

- Verilog语言中提供了4种类型的循环语句：while，for，forever，repeat。
- for语法：for(循环变量初值；循环结束条件；变量增值)

```
begin
    执行语句;
end
```

  – 循环语句属于过程语句，只能在always和initial块中使用；循环控制变量必须为整数型integer；
  – 注意与c语言中for的区别，变量增值不支持k++形式，只能k=k+1；

4.用Verilog设计译码器

例：采用for语句实现 2-to-4 decoder

```
module dec2to4 (
    input   [1:0] w,
    input   en,
    output  reg  [0:3] y
);
    integer k;
    always @(w, en)
      for(k=0;k<=3;k=k+1)
        if ((w==k))&&(en==1))
            y[k]=1;
        else
            y[k]=0;
endmodule
```

4.用Verilog设计译码器

更直观的方式实现 3-to-8 decoder（推荐）：

理解：

- 输出 out 的第 0 比特如何生成呢？当输入 in等于 0 的时候置 1，否则置 0。
- 如果实现4-to-64等更高规格的译码器，可以借助于generate 语句改善编码效率。

```
module dec3to8 (
    input   [2:0] in,
    output  [7:0] out
);
    assign   out[0] = (in == 3'd0);
    assign   out[1] = (in == 3'd1);
    assign   out[2] = (in == 3'd2);
    assign   out[3] = (in == 3'd3);
    assign   out[4] = (in == 3'd4);
    assign   out[5] = (in == 3'd5);
    assign   out[6] = (in == 3'd6);
    assign   out[7] = (in == 3'd7);
endmodule
```

5.奇偶校验电路

- 校验码：计算系统中，数据的读写、存储及传输过程中，可能会产生错误。数据校验码是带有发现某些错误，甚至具有能自动改错能力的数据编码方法。
- 校验原理：将有效代码和增加的校验位一起，按约定的校验规律进行编码，即：加进一些冗余码，使合法编码出现某些错误时，就成为非法编码。
- 码距：指任意两个合法码间至少有几个不同。
  - 若码距为1，则当发生错误时仍是合法码，无查错能力.
  - 合理地增大码距，可以提高查错能力，但码的位数越多，数据存储的容量或数据传送的数量就越大.

5.奇偶校验电路

- 奇偶校验原理
  - 增加1位校验码，使码距由1增加到2。
  - 格式： 1位校验码 + n位数据
  - 特点：实现简单、开销最小，但只能发现数据中一位（或奇数个位）出错的情况，且不能纠错。
  - 例如：奇校验： 1 10110111、0 01100001
    偶校验： 0 11110011、1 00001000

5.奇偶校验电路

实现：

校验产生与校验电路可以采用异或门实现：

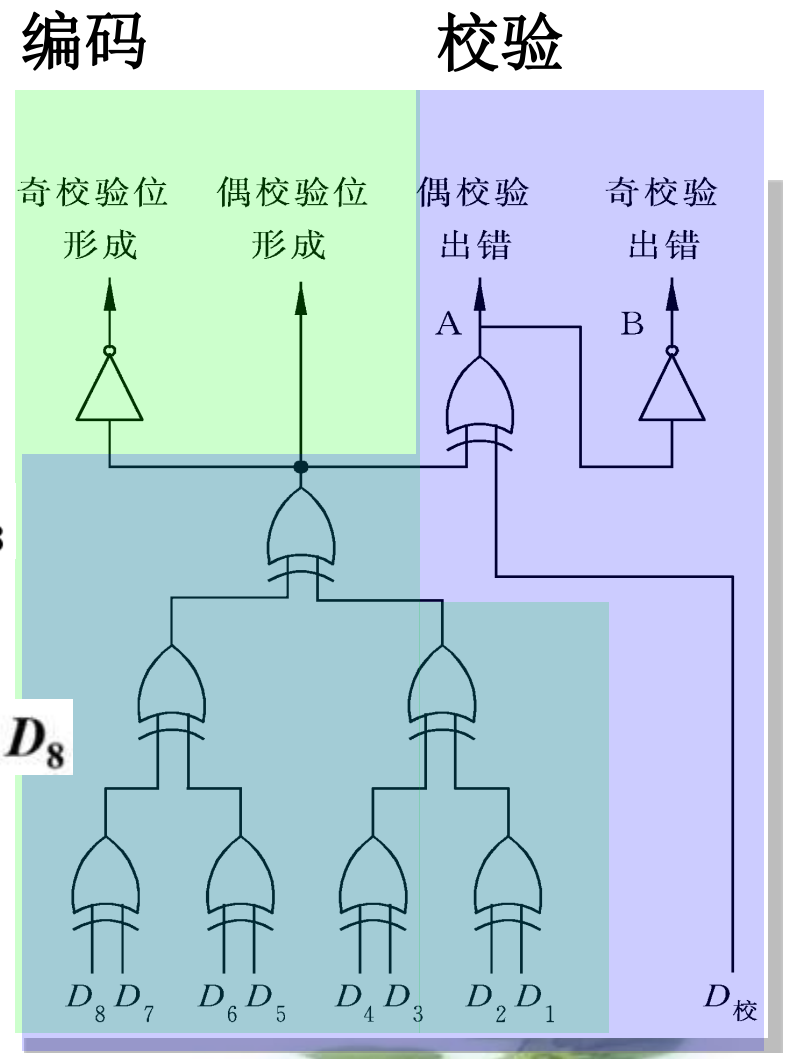
- 数据：8位数据 D_8~D_1；
- 偶校验位产生：

$$P=D_1\oplus D_2\oplus D_3\oplus D_4\oplus D_5\oplus D_6\oplus D_7\oplus D_8$$

- 接收端校验信号：

$$C=P\oplus D_1\oplus D_2\oplus D_3\oplus D_4\oplus D_5\oplus D_6\oplus D_7\oplus D_8$$

- 若C=0，则表明接收端的校验正确；若C=1，则出错。



5.奇偶校验电路

例子：

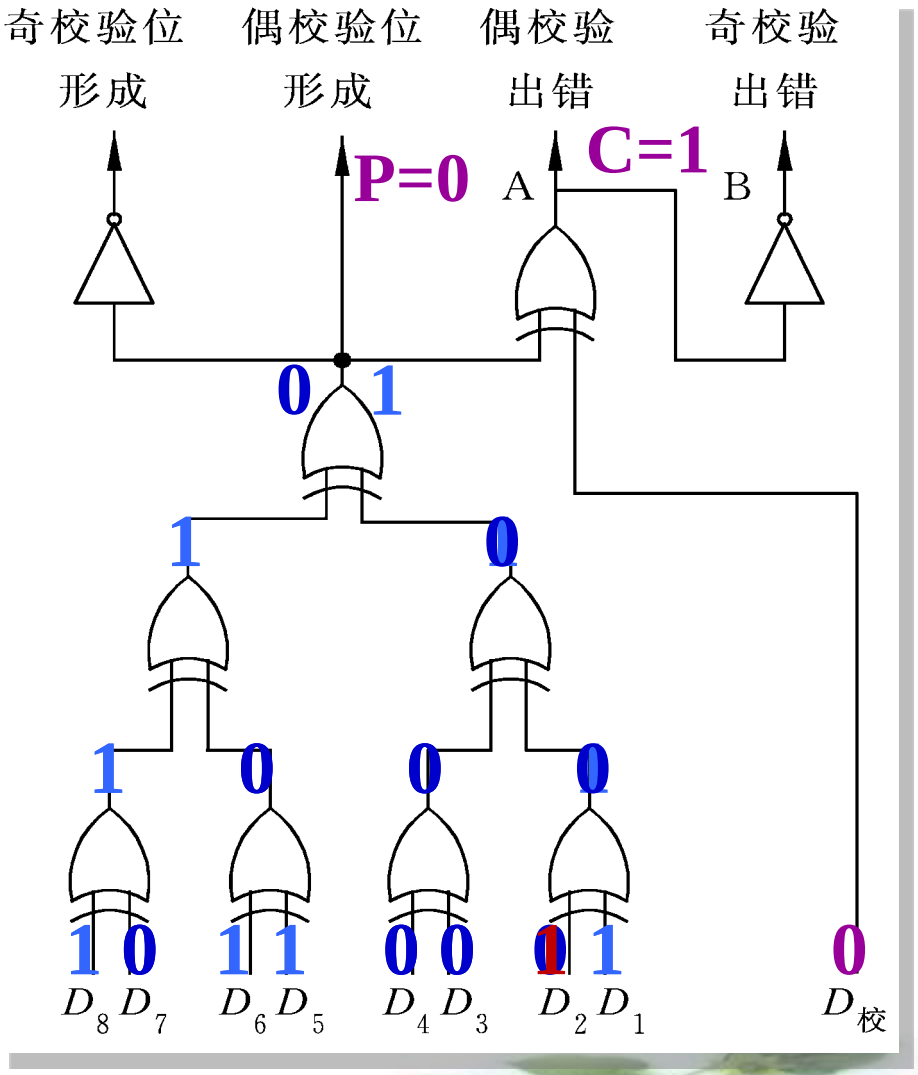
编码（发送方）：

- 数据：10110001
- 偶校验：010110001

校验（接收方）：

- 偶校验：010110011

结果：校验出错



5.奇偶校验电路

- 例：ASCII码采用7位模式表示字符的编码方式，为了用1个字节表示一个字符，第8位b_7通常置0。若需要传输该字符，则b_7可用作校验位。采用Verilog设计实现偶校验。

```
module parity (
    input  [7:0] X ,
    output [7:0] Y
);
    assign  Y =  {^X[6:0], X[6:0]};

endmodule
```

图 4.56. Verilog code for parity

本章作业：

- **4.1（或4.2），4.3（或4.4），4.5，4.8，4.11；**
- **4.16，4.18，4.19，4.21，4.22，4.25，**
- **4.26，4.27，**
- **4.29，4.30，4.32，4.33**